Universidad de la Fuerzas Armadas – ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis y Diseño de Software- NRC 30723

Guía: PlantUML, casos de uso, análisis CRUD de estudiantes

Grupo: 5o

Integrantes:

Steven Egas

Alejandro Obando

Juan Quimbiulco

Docente: Ing. Jenny Ruiz

Fecha: 28 de abril del 2026

1. Introducción

El análisis orientado a objetos propone una forma sistemática de comprender los sistemas de software antes de programarlos. Su valor principal radica en que obliga al equipo de desarrollo a pensar en qué necesita el usuario, qué estructura de objetos soporta esas necesidades y cómo esos objetos colaboran entre sí para completar cada tarea.

El presente informe documenta el análisis completo de un sistema de gestión de estudiantes implementado como un CRUD (Create, Read, Update, Delete), utilizando PlantUML como herramienta de modelado. El caso base maneja tres datos fundamentales por cada estudiante: ID, Nombre y Edad. Las operaciones disponibles son Agregar, Actualizar, Eliminar y Mostrar Todo.

El informe sigue la ruta de análisis definida en la Unidad 1 de la asignatura, articulando tres modelos sucesivos: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis. Entre ellos se mantiene una trazabilidad explícita que parte desde los requisitos funcionales de nivel 0 y llega hasta la interacción precisa entre objetos en tiempo de ejecución.

El objetivo no es implementar la interfaz gráfica ni elegir tecnologías específicas, sino demostrar coherencia entre lo que el sistema debe hacer, los objetos que lo componen y la secuencia de mensajes que ejecuta cada funcionalidad.

2. Desarrollo

2.1 Requisitos Funcionales de Nivel 0

El punto de partida del análisis es la identificación de los requisitos funcionales de nivel 0. Estos requisitos describen las funcionalidades visibles para el actor sin entrar en detalles de implementación. Para el CRUD de Estudiante se identificaron cinco requisitos, los cuatro correspondientes a las operaciones principales y uno adicional que representa la validación de datos, responsabilidad del propio sistema.

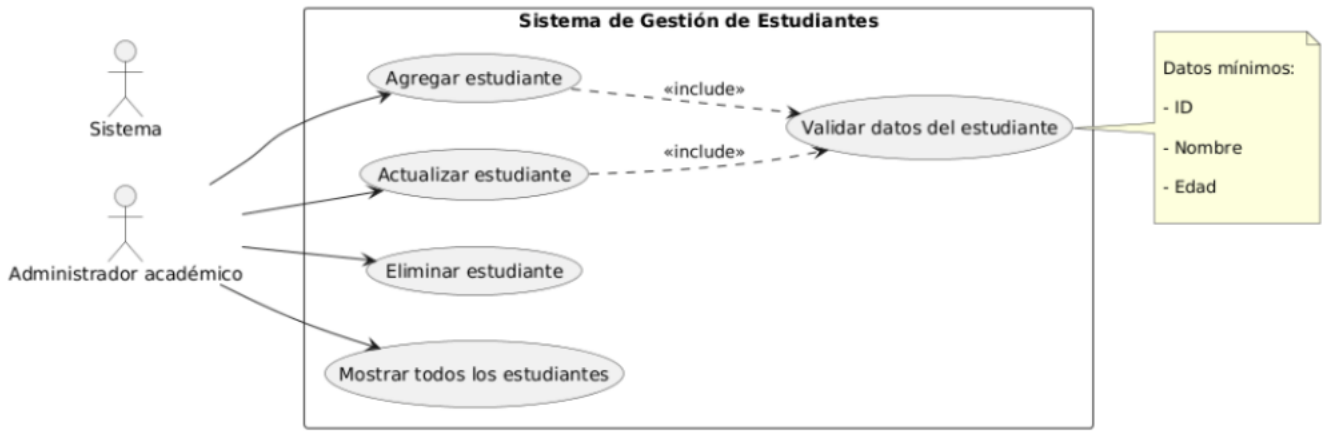
Código	Requisito Funcional (Nivel 0)	Actor Principal	Caso de Uso	Prioridad
RF-01	El sistema debe permitir agregar un estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante	Alta
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente.	Administrador académico	Actualizar estudiante	Alta
RF-03	El sistema debe permitir eliminar un estudiante existente cuando corresponda.	Administrador académico	Eliminar estudiante	Media
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados.	Administrador académico	Mostrar todos los estudiantes	Alta
RF-05	El sistema debe validar los datos obligatorios antes de agregar o actualizar.	Sistema	Validar datos del estudiante	Alta

El RF-05 (Validar datos del estudiante) no está asociado a una acción directa del actor, sino que se incorpora como comportamiento obligatorio del sistema antes de ejecutar las operaciones de escritura (Agregar y Actualizar). Esta decisión se refleja en el diagrama de casos de uso mediante la relación <<include>>.

2.2 Diagrama de Casos de Uso de Análisis

El diagrama de casos de uso representa las funciones observables desde la perspectiva del actor externo. Se identificó un actor primario (Administrador académico) y un actor de sistema que modela el comportamiento interno de validación. El límite del sistema delimita claramente qué funcionalidades pertenecen a la solución.

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema de Gestión de Estudiantes" {
  usecase "Agregar estudiante" as UC1
  usecase "Actualizar estudiante" as UC2
  usecase "Eliminar estudiante" as UC3
  usecase "Mostrar todos los estudiantes" as UC4
  usecase "Validar datos del estudiante" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>
@enduml
```



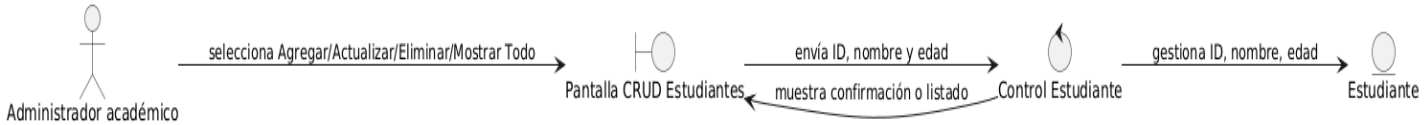
La relación <<include>> entre UC1/UC2 y UC5 está justificada funcionalmente: la validación no es opcional ni condicional, sino obligatoria cada vez que se intenta agregar o actualizar un estudiante. No se incorporaron relaciones <<extend>> porque ninguna funcionalidad del CRUD tiene comportamiento opcional condicionado que extienda a otro caso de uso.

Caso de Uso	Actor	Precondición	Flujo Principal Resumido	Resultado Esperado
Agregar estudiante	Administrador académico	El ID no debe existir previamente.	Ingresar ID, Nombre y Edad → validar datos → verificar ID único → guardar estudiante.	Estudiante agregado correctamente y visible en la tabla.
Actualizar estudiante	Administrador académico	El estudiante debe existir.	Ingresar ID y datos nuevos → validar datos → buscar estudiante → guardar cambios.	Datos del estudiante actualizados.
Eliminar estudiante	Administrador académico	El estudiante debe existir y confirmarse la eliminación.	Seleccionar o ingresar ID → buscar estudiante → eliminar registro.	Estudiante eliminado o retirado de la tabla.
Mostrar todos los estudiantes	Administrador académico	Pueden existir cero o más estudiantes registrados.	Solicitar listado → recuperar registros → mostrar tabla.	Tabla actualizada con ID, Nombre y Edad.
Validar datos del estudiante	Sistema	Se invoca desde Agregar o Actualizar.	Comprobar que ID, Nombre y Edad no estén vacíos y que Edad sea numérica y mayor a 0.	Datos validados; se impide guardar si hay errores.

2.3 Puente hacia clases de análisis: frontera, control y entidad

En el diagrama que sirve como puente hacia clases de análisis podemos identificar como el administrador del sistema interactúa con el sistema, usando las diferentes funcionalidades que tiene el sistema.

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema de Gestión de Estudiantes" {
  usecase "Agregar estudiante" as UC1
  usecase "Actualizar estudiante" as UC2
  usecase "Eliminar estudiante" as UC3
  usecase "Mostrar todos los estudiantes" as UC4
  usecase "Validar datos del estudiante" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>
@enduml
```



2.4 Diagrama de Clases de Análisis

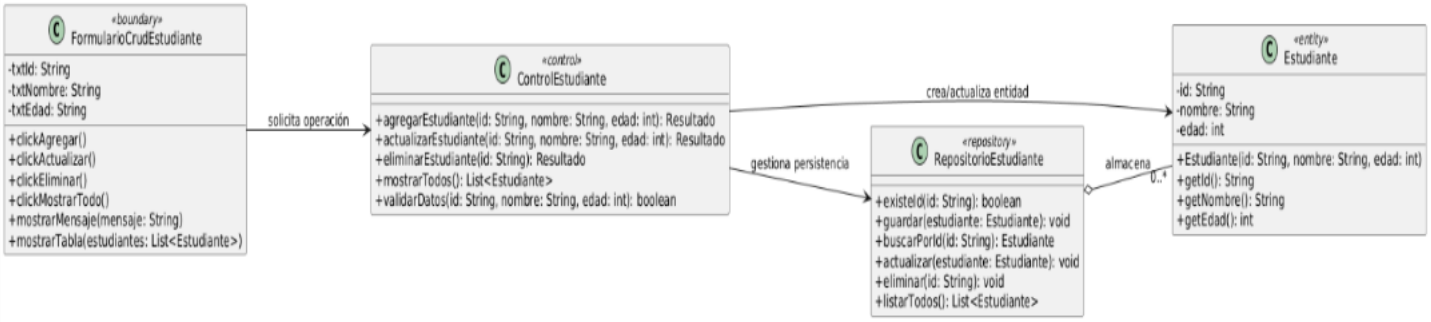
A partir de los casos de uso se derivan las clases de análisis. El modelo usa cuatro estereotipos que permiten separar claramente las responsabilidades:

- Boundary (frontera): representa la pantalla o interfaz con la que interactúa el actor.
- Control: coordina la lógica del caso de uso, actuando como intermediario entre frontera y entidad.
- Entity (entidad): modela los datos del dominio que el sistema persiste.
- Repository (repositorio): gestiona la colección de entidades, abstrayendo la persistencia.

Estereotipo	Clase	Responsabilidad	Relación con CRUD
<<boundary>>	FormularioCrudEstudiante	Recibe datos del actor y muestra resultados.	Pantalla con campos ID, Nombre, Edad y botones CRUD.
<<control>>	ControlEstudiante	Coordina la lógica de cada caso de uso.	Valida datos, decide si agrega, actualiza, elimina o consulta.
<<entity>>	Estudiante	Representa la información del dominio.	Contiene los atributos id, nombre y edad.
<<repository>>	RepositorioEstudiante	Conserva y recupera registros.	Permite guardar, buscar, actualizar, eliminar y listar estudiantes.

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioCrudEstudiante" <<boundary>> {
  - txtId: String
  - txtNombre: String
  - txtEdad: String
  + clickAgregar()
  + clickActualizar()
  + clickEliminar()
  + clickMostrarTodo()
  + mostrarMensaje(mensaje: String)
  + mostrarTabla(estudiantes: List<Estudiante>)
}

class "ControlEstudiante" <<control>> {
  + agregarEstudiante(id, nombre, edad): Resultado
  + actualizarEstudiante(id, nombre, edad): Resultado
  + eliminarEstudiante(id): Resultado
  + mostrarTodos(): List<Estudiante>
  + validarDatos(id, nombre, edad): boolean
}

class "Estudiante" <<entity>> {
  - id: String
  - nombre: String
  - edad: int
  + getId(): String
  + getNombre(): String
  + getEdad(): int
```

```
}

class "RepositorioEstudiante" <<repository>> {
  + existeId(id): boolean
  + guardar(estudiante): void
  + buscarPorId(id): Estudiante
  + actualizar(estudiante): void
  + eliminar(id): void
  + listarTodos(): List<Estudiante>
}

FormularioCrudEstudiante --> ControlEstudiante
ControlEstudiante --> Estudiante
ControlEstudiante --> RepositorioEstudiante
RepositorioEstudiante o-- "0..*" Estudiante
@enduml
```



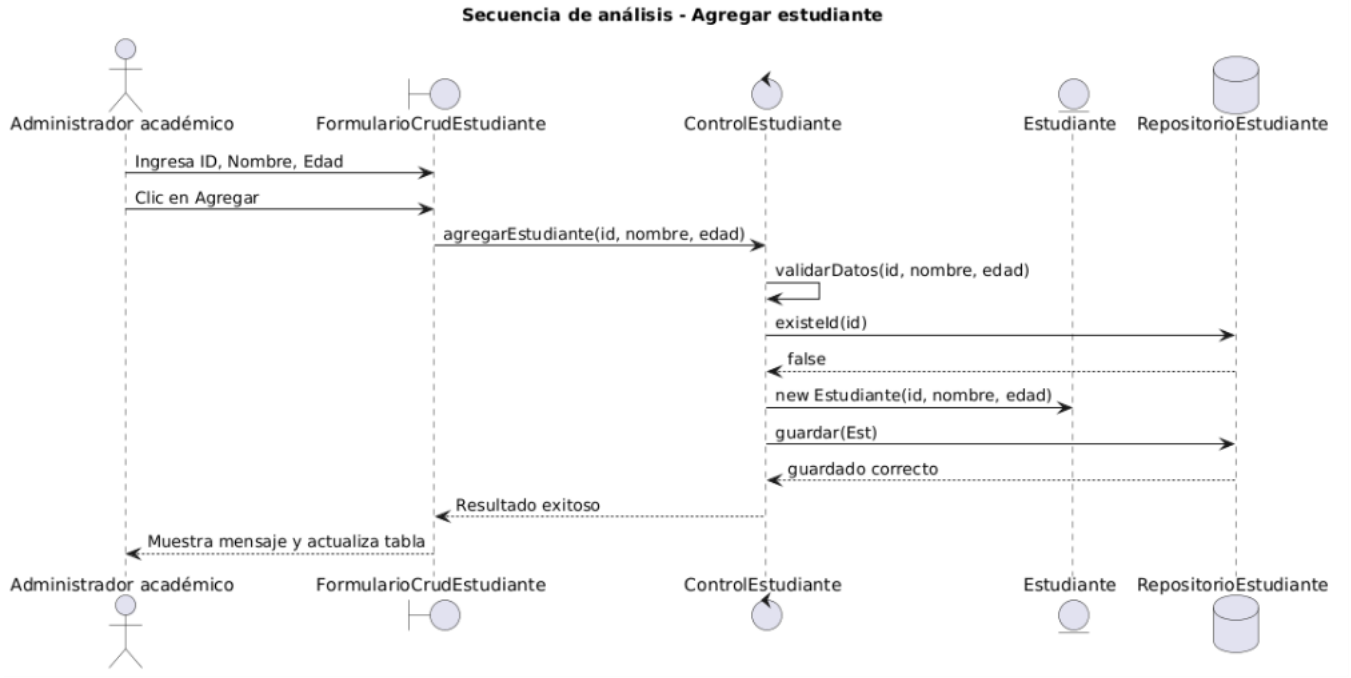
2.5 Diagramas de Secuencia de Análisis

Los diagramas de secuencia muestran cómo los objetos identificados en el diagrama de clases colaboran en tiempo de ejecución para completar cada caso de uso. Se elaboró un diagrama independiente por cada operación del CRUD para mantener la claridad del análisis.

Seq-01: Agregar estudiante

```
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
```

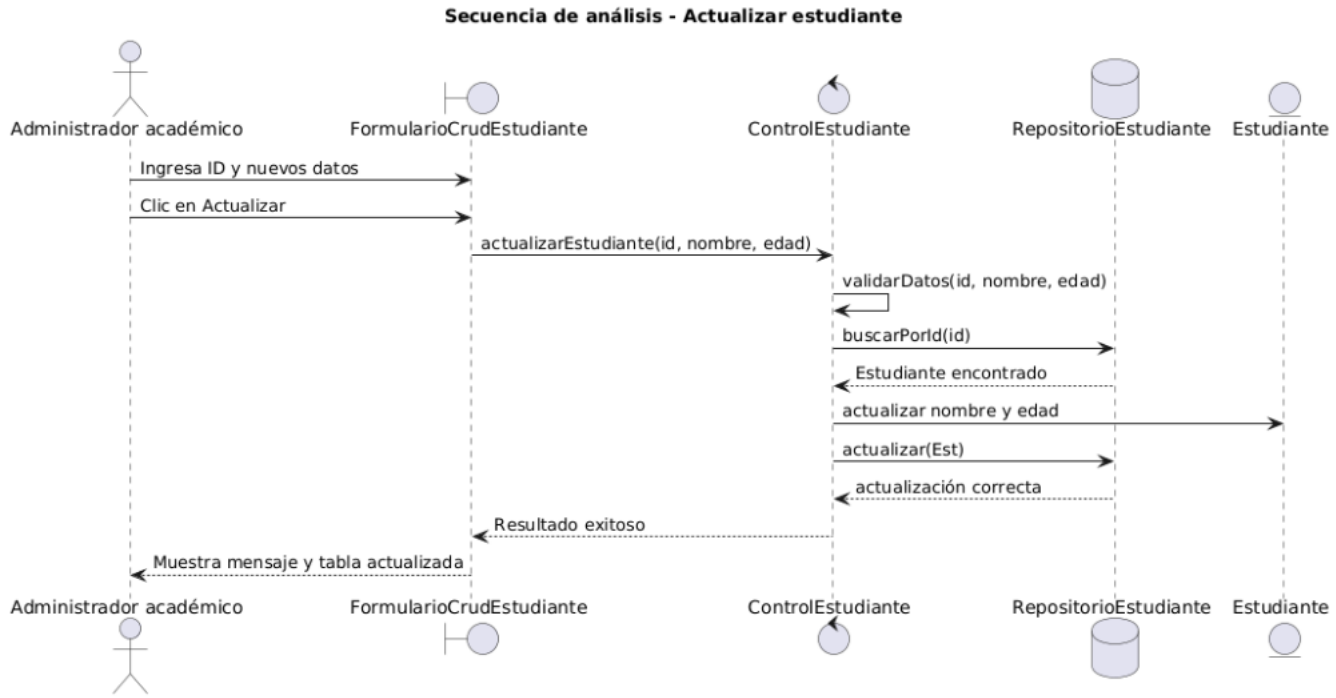
```
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```



Seq-02: Actualizar estudiante

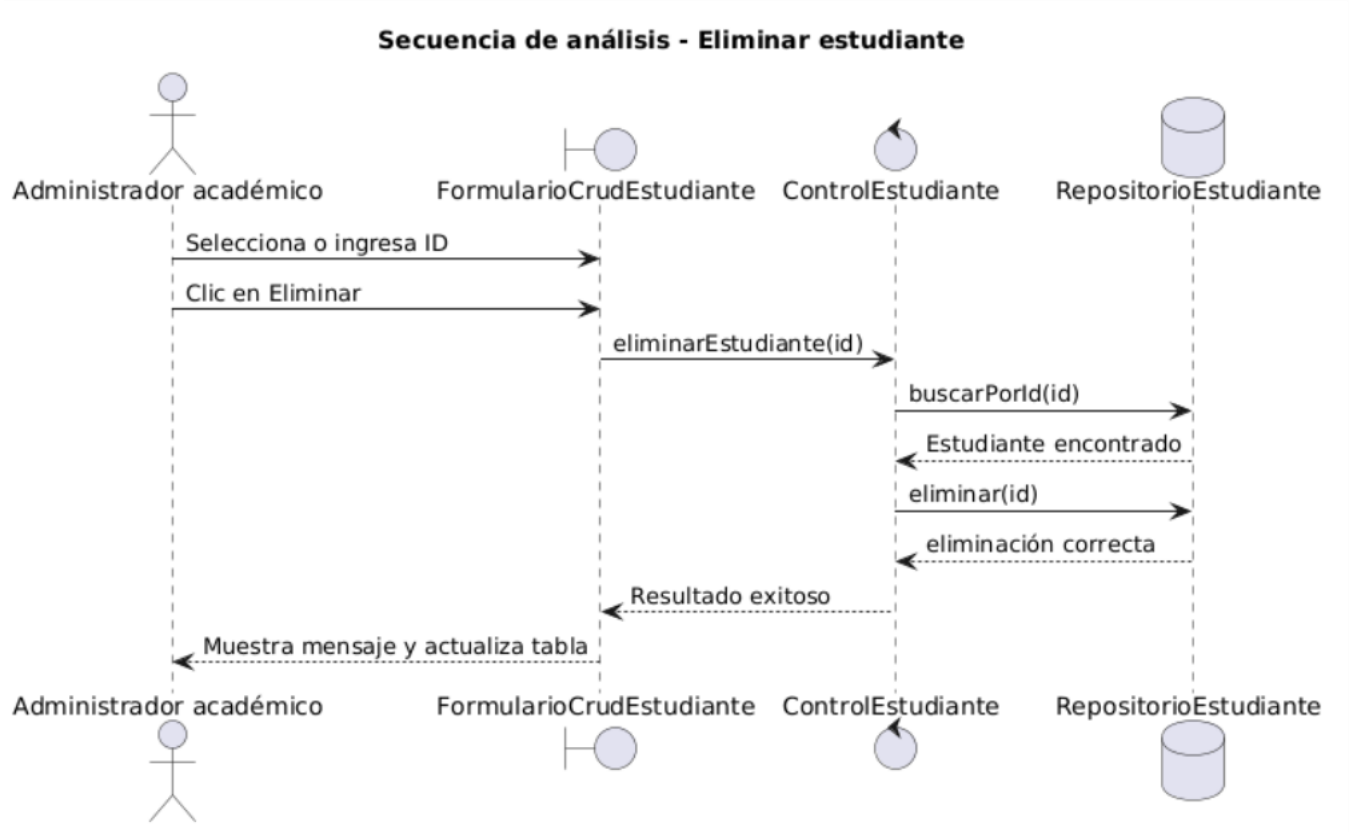
```
@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
```

```
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```



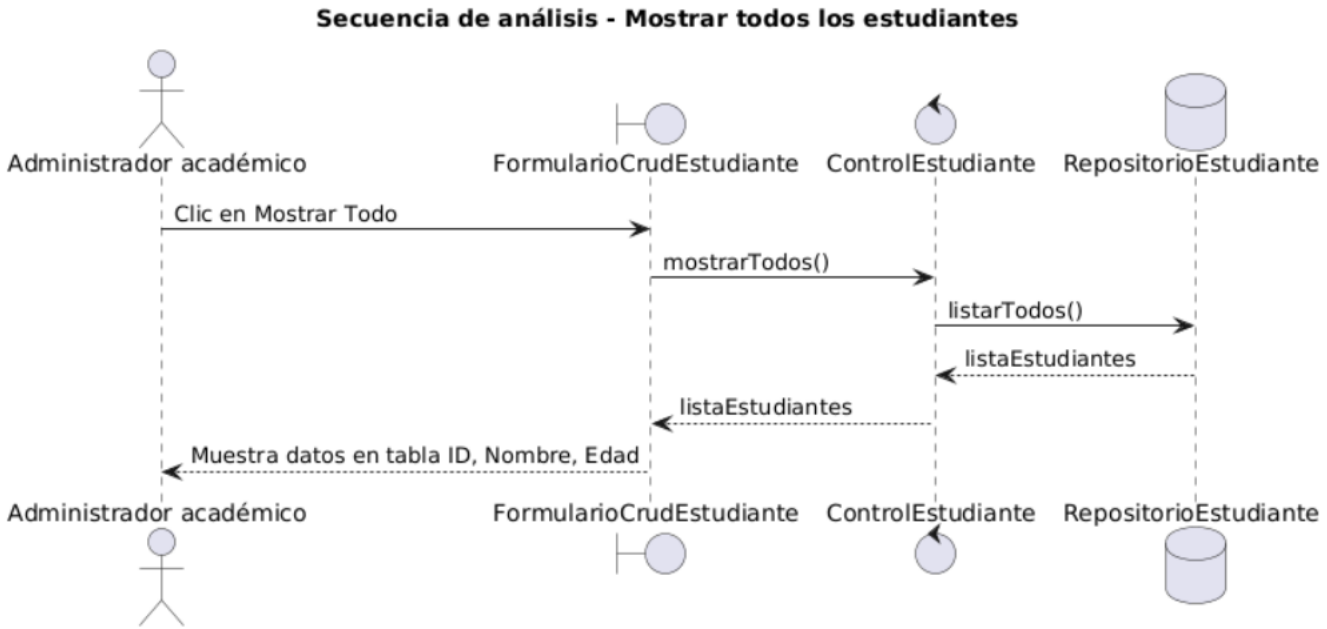
Seq-03: Eliminar estudiante

```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```



Seq-04: Mostrar todos los estudiantes

```
@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
```



```
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```

3. Matriz de Trazabilidad RF → CU → Clases → Secuencias

La matriz de trazabilidad es el instrumento que garantiza la coherencia del análisis completo. Permite verificar que cada requisito funcional está cubierto por un caso de uso, que ese caso de uso cuenta con clases de análisis que lo soportan y que existe un diagrama de secuencia que muestra cómo los objetos colaboran para ejecutarlo. Ningún elemento del modelo debe aparecer sin su correspondiente requisito funcional de origen.

RF	Caso de Uso	Actor	Prioridad	Clases Involucradas	Diagrama de Secuencia	Criterio de Aceptación
RF-01	Agregar estudiante	Administrador académico	Alta	FormularioCrud → ControlEstudiante → Estudiante → RepositorioEstudiante	Seq-01: Agregar	Sistema guarda estudiante con ID único, Nombre obligatorio y Edad válida.
RF-02	Actualizar estudiante	Administrador académico	Alta	FormularioCrud → ControlEstudiante → Estudiante → RepositorioEstudiante	Seq-02: Actualizar	Sistema modifica Nombre y/o Edad tras validar campos obligatorios.
RF-03	Eliminar estudiante	Administrador académico	Media	FormularioCrud → ControlEstudiante → RepositorioEstudiante	Seq-03: Eliminar	Sistema elimina o marca como eliminado al estudiante seleccionado.
RF-04	Mostrar todos los estudiantes	Administrador académico	Alta	FormularioCrud → ControlEstudiante → RepositorioEstudiante	Seq-04: Mostrar Todo	Sistema presenta tabla con columnas ID, Nombre y Edad.
RF-05	Validar datos del estudiante	Sistema	Alta	ControlEstudiante	Incluido en Seq-01 y Seq-02	Sistema impide guardar con ID vacío, nombre vacío o edad inválida.

La trazabilidad completa confirma que el modelo no contiene casos de uso huérfanos, clases sin responsabilidad asignada ni secuencias que referencien operaciones no declaradas en el diagrama de clases. RF-05 aparece integrado en las secuencias Seq-01 y Seq-02 a través del mensaje

validarDatos(), lo que evidencia que su inclusión mediante <<include>> en el diagrama de casos de uso tiene correlato directo en el modelo dinámico.

ID	Caso de Uso	Mensajes Principales	Clases Participantes	Resultado del Flujo
Seq-01	Agregar estudiante	clickAgregar → agregarEstudiante → validarDatos → existeId → new Estudiante → guardar → Resultado exitoso	UI, ControlEstudiante, RepositorioEstudiante, Estudiante	Estudiante guardado y mensaje de confirmación.
Seq-02	Actualizar estudiante	clickActualizar → actualizarEstudiante → validarDatos → buscarPorId → actualizar → Resultado exitoso	UI, ControlEstudiante, RepositorioEstudiante, Estudiante	Datos modificados y tabla actualizada.
Seq-03	Eliminar estudiante	clickEliminar → eliminarEstudiante → buscarPorId → eliminar → Resultado exitoso	UI, ControlEstudiante, RepositorioEstudiante	Estudiante eliminado y tabla actualizada.
Seq-04	Mostrar todos los estudiantes	clickMostrarTodo → mostrarTodos → listarTodos → listaEstudiantes → mostrarTabla	UI, ControlEstudiante, RepositorioEstudiante	Tabla completa presentada al actor.

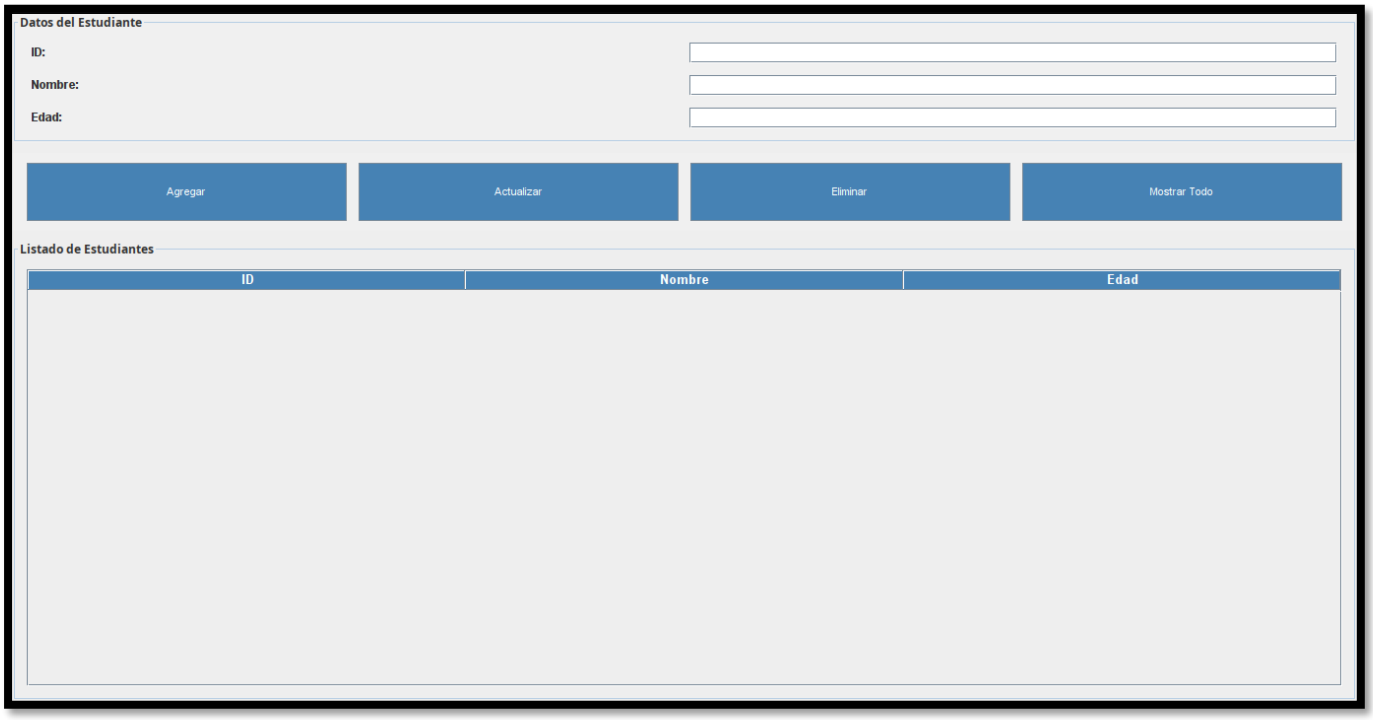
4. Conclusión

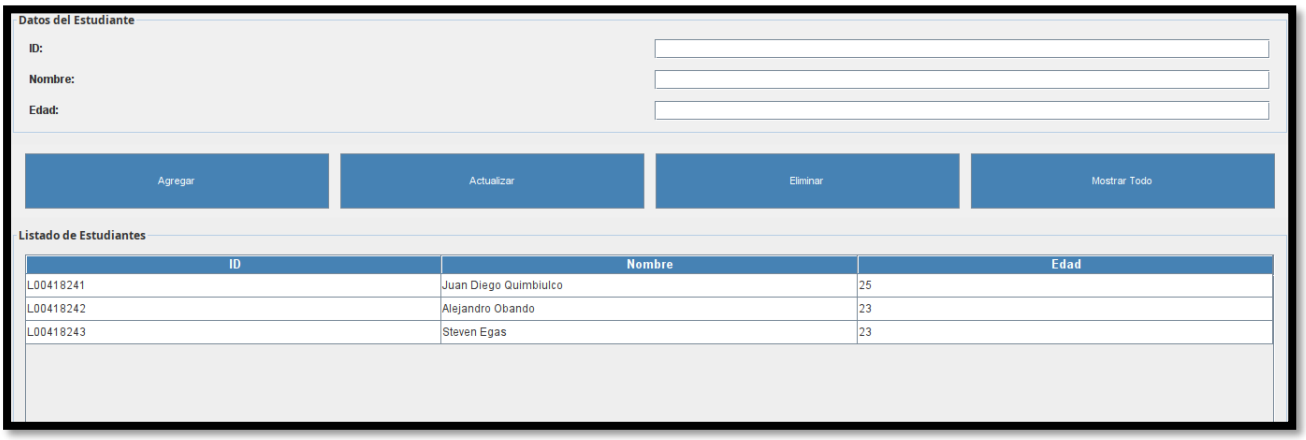
La conclusión técnica del CRUD de Estudiante evidencia que el análisis se construye de manera coherente a partir de los casos de uso, los cuales actúan como eje organizador del modelo. Estos casos de uso, como Agregar estudiante, Actualizar estudiante, Eliminar estudiante y Mostrar todo, definen con claridad las necesidades del actor y sirven como base para derivar tanto las clases de análisis como los diagramas de secuencia.

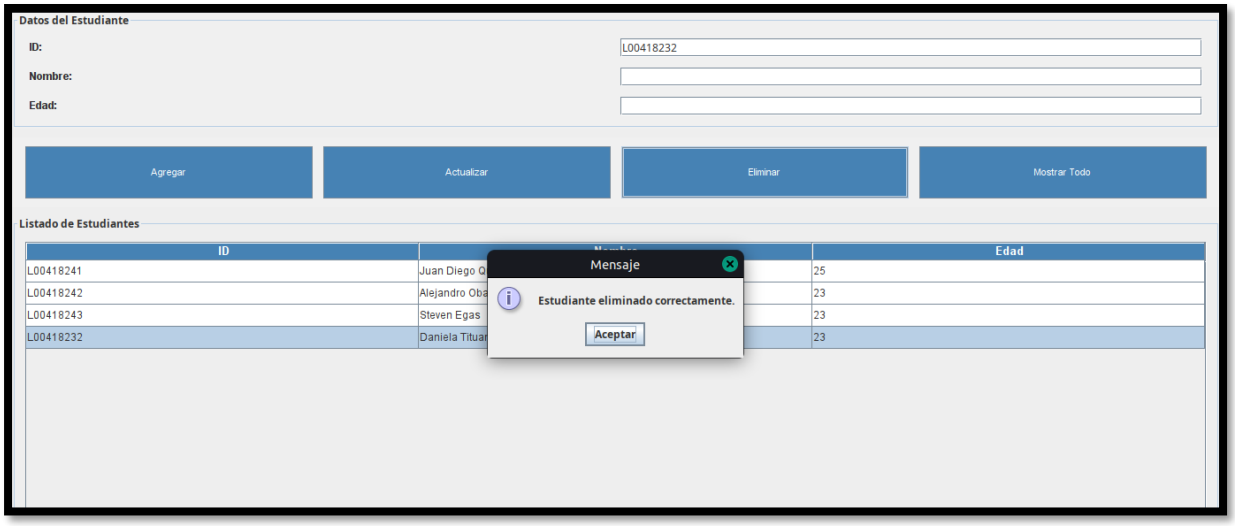
Por ejemplo, en Agregar estudiante y Actualizar estudiante, la inclusión de la validación de datos como un comportamiento obligatorio se refleja posteriormente en la lógica del control y en las secuencias, donde siempre se valida antes de guardar o modificar información. En Eliminar estudiante, se observa la correspondencia entre el caso de uso y la secuencia al verificar primero la existencia del registro antes de eliminarlo. En Mostrar todo, la funcionalidad se traduce directamente en la consulta y visualización de la lista de estudiantes, manteniendo consistencia entre lo definido y lo ejecutado.

De esta forma, las clases de análisis representan los objetos necesarios para cumplir cada caso de uso, mientras que los diagramas de secuencia permiten comprobar que las interacciones entre estos objetos responden exactamente a las funcionalidades planteadas. La trazabilidad entre requisitos, casos de uso, clases y secuencias permite validar que el modelo es consistente, completo y alineado con las necesidades del sistema, proporcionando una base sólida para el diseño posterior.

5. Anexos


Datos del Estudiante
ID:
Nombre:
Edad:
Agregar
Actualizar
Eliminar
Mostrar Todo
Listado de Estudiantes
ID
Nombre
Edad


Datos del Estudiante
ID:
Nombre:
Edad:
Agregar
Actualizar
Eliminar
Mostrar Todo
Listado de Estudiantes
ID
Nombre
Edad
L00418241
Juan Diego Quimbiulco
25
L00418242
Alejandro Obando
23
L00418243
Steven Egas
23


Datos del Estudiante
ID:
L00418232
Nombre:
Edad:
Agregar
Actualizar
Eliminar
Mostrar Todo
Listado de Estudiantes
ID
Edad
L00418241
Juan Diego O
25
L00418242
Alejandro Oba
23
L00418243
Steven Egas
23
L00418232
Daniela Tituar
23
Mensaje
Estudiante eliminado correctamente.
Aceptar